COMPUTER PROGRAMMING

UNIT – V

Syllabus:

Files – operations on a file – Random access to files – command line arguments. Introduction to preprocessor – Macro substitution directives – File inclusion directives – conditional compilation directives – Miscellaneous directives.

Part – A (2 marks)

1. Discuss the different ways of opening a file.(May 2014)

There are different ways of opening a file, they are:

"r"	open the file for read only.
"w"	open the file for writing only.
"a"	open the file for appending data to it.
r+	Read and write
w+	Write and read
"a+"	Opens existing file for reading and writing.

2. How can files be randomly accessed? (May 2014)

To access a particular part of a file randomly and not in reading the other parts. This is done by using the I/O library functions, namely

(i) fseek()
(ii) ftell()
(iii)rewind()

3. Differentiate between ftell() and fseek() functions.(Jan 2015)

ftell()	fseek()
i)This function returns the value of the current pointer in the file.	i)This function is used to move the file position to a desired location within the file.
ii)Gets the total size of the file after moving the file pointer at the end of the	ii)It is used to write data into the file at desired location.

file.	
Syntax : ftell(fptr);	**Syntax** : fseek(file_pointer, offset, position);

4. When does a programmer use "#include" directive?(Jan 2015)

(i) This directive is used for including one or more files in the program.

(ii) It instructs the compiler to make a copy of specified file to be included in place of the directive.

Example :#include<stdio.h> ⟶contains standard I/O functions in directories

5. Briefly discuss about the purpose of C preprocessor.(Jan 2016)

(i) C preprocessor is a collection of special statements called directives that are executed at the beginning of the compilation process.

(ii) It is a separate program invoked by the compiler as the first part of translation.

6. Give the conditional pre-processing directives. (May 2016)

Directives permit certain segment of source code to be selectively compiled.

The Conditional directives are

1. **#if**
2. **#else**
3. **#elif**
4. **#endif**
5. **#ifdef**
6. **#ifndef**

7. Write any eight built-in functions for file manipulation. (May 2016)

There are eight operations on files. They are

1) putc()
2) getc()
3) getw()
4) putw()
5) fscanf()
6) fprintf()
7) fread()
8) fwrite()

8. Differentiate between fopen and fclose functions. (Nov 2016)

fopen()	fclose()
This function is used to open a file to perform operations such as reading, writing, etc.	This function closes the file that being pointed by the file pointer fp.
In a c program, we declare a file pointer and use fopen().	In a c program, we close a file using fclose().
Syntax : FILE *fp; fp=fopen("filename","mode");	**Syntax :** fclose(fp);

9. When does the programmer use the #define directive. (Nov 2016)

(i) The #define directive is used to define values or macros that are used by the pre-processor to manipulate the program source code before it is complied.

(ii) It is also used to define symbolic constant and assign value.

Syntax :#define identifier set-of-characters

where identifier also known as "macro name" or "macro template".

10. What is the purpose of #ifndef? (May 2017)

(i) #ifndef – if not defined directive

(ii) In the C Programming Language, the #ifndef directive allows for conditional compilation.

(iii) The preprocessor determines if the provided macro does **not** exist before including the subsequent code in the compilation process.

11. Give the syntax of fopen().(May 2017)

The syntax of fopen() is :

FILE *fp;

Fp = fopen("filename", "file_mode");

- First statement declares the variable fp as a "pointer to the data type FILE".
- Second statement opens the file named filename and assigns an identifier to the FILE type pointer fp.

12. Name any four preprocessor directives. (Nov 2017)

The types of preprocessor directives are

(i) File inclusion directive
(ii) Macro substitution directive
(iii) Conditional directive
(iv) Miscellaneous directive

13. What are the operations on a file? (Nov 2017)

The **basic operation** performed on a file includes:

(i) Naming a File
(ii) Opening a File
(iii) Writing data into a File
(iv) Reading data from a file
(v) Closing a File

14. Distinguish between scanf() and fscanf() functions. (May 2018)

scanf()	**fscanf()**
This function is used to read formatted input from stdin in c language.	This function is used to read the formatted input from the given stream in c language.
It returns the whole number of characters written in it otherwise, returns a negative value.	It returns zero, if unsuccessful. Otherwise, it returns the input string, if successful.
Syntax: scanf("control string", argl, arg2,…,argn);	**Syntax:** fscanf(fp, "control string", list);

15. Write any two examples of macro substitution directives. (May 2018)

(i) Macro definition is the second type of preprocessor directives.
(ii) The #define directive is used here.

Example1: *#define A 10*

The **#define A 10** macro substitutes 'A' with 10 in all occurrences in the source program.

Example2 :

```
#define   exp if(10>5)
#define  Trueprintf("Greater");
#define False else printf("lesser");
```

To build the above macro as a statement like

exp True False

gives

```
if(10>5)
printf("Greater");
else
printf("lesser");
```

16. Why header files are included in C program? (Nov 2018)

(i) Header file is a file that contains function declaration and macro definition for c in-built library functions.

(ii) It allows us to put declarations in one location and then import them wherever we need.

(iii) This can save a lot of typing in multi-file programs.

17. Differentiate between Append and Write modes in files. (May 2019)

APPEND MODE	WRITE MODE
Append mode Is used to add data to the existing data of file.	In write mode, the content which was previously present in the file will be overwritten.
The cursor is positioned at the end of the present data.	The file is reset, resulting in deletion of any data already present in the file.
Keeps the original content of the file.	Destroys the file contents.

18. What are command line arguments. (May 2019)

- An executable program that performs a specific task for operating system is called as **command.**
- The commands are issued from the prompt of operating system, some arguments are associated with the commands, and hence these arguments are called as **command line arguments.**
- The main() function receives two arguments and they are
  - **(i) argc**
  - **(ii) argv**

19. What is macro in C language? (Nov 2019)

- Macro is a segment of code.
- It starts with #define.
- Whenever the macro name is used in the program it will replace with its values.

  Syntax : #define macro_name value.

20. What are the different file operation modes? (Nov 2019) (Sep 2020)

There are different file operation modes, they are:

File Type	Modes of File Operations
"r"	open the file for read only.
"w"	open the file for writing only.
"a"	open the file for appending data to it.
r+	Read and write
w+	Write and read
a+	Opens existing file for reading and writing.

21. Mention the various rules for defining preprocessor. (Sep 2020)

Rules for defining preprocessor

1. All preprocessor directives begin with **sharp sign**. i.e**.,(#)**
2. It starts in first column.
3. It should not terminate with **semicolon**.
4. It may appear at any place in the source file. i.e., Outside the function, inside the function or inside compound statements.
5. It does not require semicolon at the end.

Part – B (11 marks)

1. What are pre-processor directives? Discuss in details, the three categories of C preprocessor directives with example.(Jan 2015) (May 2014) (May 2017) (Nov 2016) (Nov 2017) (May 2018) (Nov 2019) (Sep 2020)

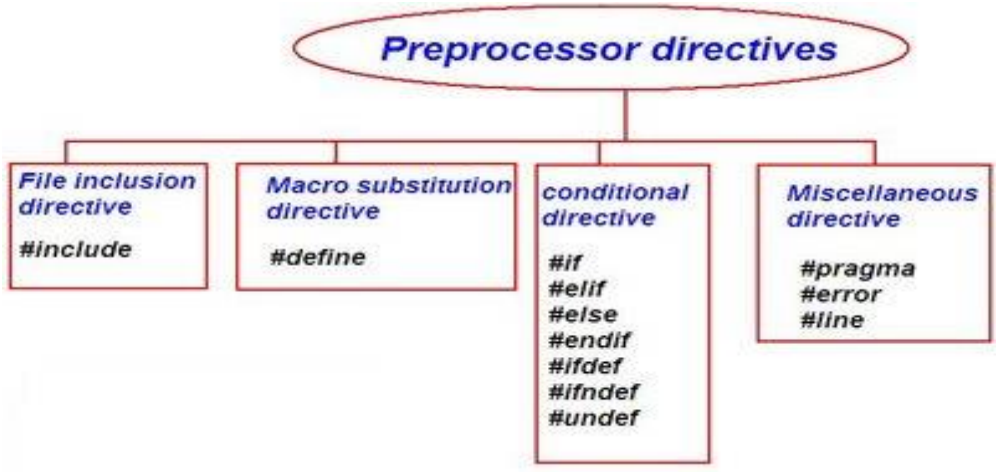
- C preprocessor is a collection of special statements called directives that are executed at the beginning of the compilation process.
- It is a separate program invoked by the compiler as the first part of translation.
- The preprocessor is a program that processes **its input data to produce output that is used as input to another program.**

TYPES OF PREPROCESSOR DIRECTIVES

The types of preprocessor directives are

1. **File inclusion directive**
2. **Macro substitution directive**
3. **Conditional directive**

4. **Miscellaneous directive**



FILE INCLUSION DIRECTIVE

It is the first type. This directive is used for including one or more files in the program. It instructs the compiler to make a copy of specified file to be included in place of the directive.

Syntax	#include<filename> #include“filename”
Description	**filename**–name of the file that can be included in our source program “**filename”-**searches file in current directories and then in standard directives. <> - search only in standard directives
Example	#include<stdio.h> ⟶ contains standard I/O functions in directories #include “loop.c”⟶ written by the user.

MACRO SUBSTITUTION DIRECTIVE

Macro definition is the second type of preprocessor directives. The #define directive is used here. It is also used to define symbolic constant and assign value.

Syntax:#define identifier set-of-characters

where identifier also known as “macro name” or “macro template”.

Rule

1. No space between # and define

2. One blank space b/n # define and identifier and b/n identifier and string
3. No semicolon (;) at the end
4. Identifier must be valid c name.

There are three different forms of macros

1. **Simple macros**
2. **Argumented macros**
3. **Nested macros**

SIMPLE MACROS

This is commonly used to define symbolic constants

Syntax:	**#define identifier set of characters**
Examples:	*#define age 20* *#define CITY "CHENNAI"* *#define g 9.8*

ARGUMENTED MACROS

The argumented macros are used to define more complex and useful form of replacements in the source program.

Syntax:

#define identifier (v1, v2 ...vn) string/integer

Example Program:

```
#define sq(n) (n*n)    /*  macro substitutions */
#define cude(n) (n*n*n)  /*  macro substitutions */
#include<stdio.h>
main()
{
int a=5, b=3, s, c;  /*local definations*/
    /* statements */
    s=sq(a);
    c=cube(b);
printf("Square value of 5 is %d\n",s);
printf("Cube value of 3 is %d\n",c);
}
```

OUTPUT

```
Square value of 5 is 25
cube value of 3 is 27
```

NESTED MACROS

The macro defined within another macro called nested macro

Syntax:	*# define identifier1* *#define identifier2 (expression with identifier1)*
Example:	*#define identifier A 5* *#define identifier B A + 2*

CONDITIONAL COMPILATION DIRECTIVE

Directives permit certain segment of source code to be selectively compiled. Directives are also called as **Conditional Compilation**.

The Conditional directives are

1. **#if**
2. **#else**
3. **#elif**
4. **#endif**
5. **#ifdef**
6. **#ifndef**

Format of #if and #endif directive.

The format of #if and #endif directive is

```
Syntax:     #if constant expression
                sequence of statements
            #endif
```

If the value of constant expression is true sequence of statements are executed. if the expression value is false compiler skip the statements.

Format of #if and #else directive

#else statement helps to terminate #if statement. If the given constant or expression is false #else block is executed.

```
Syntax:#if constant expression
```

```
                    Sequence of statements
#else
                    Sequence of statements
#endif
```

Here for **nested if statements#elif directives** can be used. The general format of #elif is

```
Syntax:     #if constant expression1
                statements
            #elif constant expression2
                sttements
                ……….
            #endif
```

Here if expression1 is true it is executed and all other expressions are skipped.
#ifdef directive is used for conditional compilation. The general form is

```
Syntax:     #ifdefmacroname
                statements
            #endif
```

MISCELLANEOUS DIRECTIVES:

There are two more pre-processor directives, thoughthey are not very commonly used. They are #undef and #pragma

1. #undef:

On some occasion, it may b desirable to cause a defined name to become 'undefined'. This can be accomplished by means of the 3undef directive.

```
Eg:
    #undef  PENTIUM
```

would cause the definition of PENTUM to be removed from the system. All subsequent #ifdef PENTUM statements became false.

2.#pragma :

A special directive we can turn on or off certain features. Pragma vary from one compiles to another.

The following table shows list of programmer command.

Sl.no	Pragma command	Description
1.	#pragma startup <function_name_1>	This directive executes function named "function_name_1"before starting of the program
2.	#pragma exit <function_name_2>	This directive executes function named "function_name_2"before termination of the program
3.	#pragma warn-rvl	If function doesn't return a value, then warnings are suppressed by this directive while compiling .
4.	#pragma warm-par	If function doesn't use passed function parameter, then warnings are suppressed.
5.	#pragma warn-rch	If non-reachable code is written inside a program, such warnings are suppressed by this directive.

4. What are command line arguments? Give the syntax and the method of execute the same. Write a command line argument to find the sum of two numbers. (May 2016) (Nov 2018)

- An executable program that performs a specific task for operating system is called as **command.**
- The commands are issued from the prompt of operating system, some arguments are associated with the commands, and hence these arguments are called as **command line arguments.**

The main() function receives two arguments and they are

1. argc
2. argv

Argument argc:

An argument **argc** counts total number of arguments passed from command prompt. It returns a value which is equal to total number of arguments passed through **main().**

Argument argv:

It is a pointer to an array of character strings which contains names of arguments. Each word is an argument.

int main (int argc, char *argv[])

- Command line argument is a parameter supplied to a program when the program is invoked. This parameter represents filename of the program that should process.

- In general, execution of C program starts from main() function. It has two arguments like **argc** and **argv**. The **argc** is an argument counter that counts the number of arguments on the command line. The **argv** is an argument vector. It represents an array of character pointer that point to command line arguments.
- The size of the array will be equal to the value of **argc**. Information contained in command line is passed to program through these arguments whenever **main** is called. The first parameter in command line is program name. Here **argv[0]** represents the program name.
- To access command line arguments, declare the main function and parameter as

```
main(intargc, char *argv[ ])
{
        …….
}
```

Program for Sum of integers using command line arguments.

```
#include<stdio.h>
#include<stdlib.h>
#include<conio.h>
void main(int argc , char *argv[])
{
int i,sum=0;
clrscr();
printf("\n\tSUM OF INTEGERS USING COMMAND LINE ARGUMENT");
printf("\n\t*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*");
   if(argc<3)
     {
     printf("\nAtleast Type 2 Numbers");
     exit(1);
     }
printf("\nThe sum is : ");
   for(i=1;i<argc;i++)
      sum = sum + atoi(argv[i]);
printf("%d",sum);
}
```

5. What do you mean by file? Write a C program to copy the content of one file to another file. (Nov 2017) (May 2016)

"A file is a collection of related information that is permanently stored on the disk and allows us to access and alter the information whenever necessary."

The **basic operation** performed on a file includes:

- Naming a File
- Opening a File

- Writing data into a File
- Reading data from a file
- Closing a File

C program to copy contents of one file to another file

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    FILE *fptr1, *fptr2;
    char filename[100], c;
   printf("Enter the filename to open for reading \n");
    scanf("%s", filename);
    fptr1 = fopen(filename, "r");
     if(fptr1 == NULL)
       {
          printf("Cannot open file %s \n", filename);
          exit(0);
       }
  printf("Enter the filename to open for writing \n");
    scanf("%s", filename);
    fptr2 = fopen(filename, "w");
       if(fptr2 == NULL)
      {
          printf("Cannot open file %s \n", filename);
          exit(0);
      }
   c = fgetc(fptr1);
    while(c != EOF)
    {
        fputc(c, fptr2);
        c = fgetc(fptr1);
    }
  printf("\nContents copied to %s", filename);
  fclose(fptr1);
  fclose(fptr2);
  return0;
}
```

Output:

```
Enter the filename to open for reading
a.txt
Enter the filename to open for writing
b.txt
Contents copied to b.txt
```

6, **Explain with suitable examples how random access can be performed in files.**

(Nov 2016)

Sequential Files are generally used in cases where the program processes the data in a sequential fashion.

Eg: counting words in a text file

Random Access File functions

To access a particular part of a file randomly and not in reading the other parts. This is done by using the I/O library functions, namely

1. **fseek()**
2. **ftell()**
3. **rewind()**

fseek() Function:

The fseek() function allows us to read from or write to any point in the stream of bytes opened by using the fopen() function. It is used to move the position of the file pointer to the desired location in the file.

Syntax	fseek(fptr, offset, position)
Description	**fptr–**file pointer of the file to be accessed. **offset–**is a negative or positive number or a variable of long integer data type to reposition the file pointer towards the backward or forward direction from the location specified by position. **position–**Current position of the file pointer.
Example:	Beginning ↓ fseek(fptr,5L,0); ↑ 5 byte

ftell() FUNCTION

Function returns the current position of the file pointer in the file.

Syntax

long intftell(fptr);

Example

a = ftell(fptr); → Returns the size of the file in bytes to the variable **a**.

rewind() FUNCTION

Function resets the file pointer **to the beginning** of the file.

Syntax rewind(fptr);

Program that uses the functions ftell and fseek.

```
#include <stdio.h>
main()
{
   FILE  *fp;
   long  n;
   char c;
   fp = fopen("RANDOM", "w");
   while((c = getchar()) != EOF)
  putc(c,fp);

  printf("No. of characters entered = %ld\n", ftell(fp));
  fclose(fp);
  fp = fopen("RANDOM","r");
   n = 0L;

   while(feof(fp) == 0)
   {
  fseek(fp, n, 0);  /*  Position to (n+1)th character */
  printf("Position of %c is %ld\n", getc(fp),ftell(fp));
      n = n+5L;
   }
 putchar('\n');
fseek(fp,-1L,2);     /*  Position to the last character */
     do
     {
      putchar(getc(fp));
     }while(!fseek(fp,-2L,1));
```

```
fclose(fp);
  }
```

Output:

```
  ABCDEFGHIJKLMNOPQRSTUVWXYZ^Z
  No. of characters entered = 26
  Position of A is 0
  Position of F is 5
  Position of K is 10
  Position of P is 15
  Position of U is 20
  Position of Z is 25
  Position of   is 30
ZYXWVUTSRQPONMLKJIHGFEDCBA
```

7. Write a program using command line argument to copy one file to another.

(May 2017)

Program to copy the content of one file to another file using command line arguments.

```
#include<stdio.h>
#include<stdlib.h>
int main(intargc,char *argv[])
{
FILE *fp1,*fp2;
inti,ch;
if(argc!=3)
{
printf("Program parameter missing");
exit(0);
}
fp1=fopen(argv[1],"r");
fp2=fopen(argv[2],"a");
while((ch=getc(fp1))!=EOF)
putc(ch,fp2);
printf("file %s is successfully copied",argv[1]);
return 0;
}
```

8. Explain in detail, the various high level I/O operations performed on files with suitable examples.(Jan 2015)

There are eight operations on files. They are

1. **putc()**
2. **getc()**
3. **getw()**
4. **putw()**

5. **fscanf()**
6. **fread()**
7. **fprintf()**
8. **fwrite()**

Single character input/output functions for files

- Once a file is opened, an Input/Output operations is performed on file, source of the functions which facilitate **single character Input/Output for file getc() &putc().**
- The single character Input function fgetc() receives a single argument, a file pointer for the file from which a character is read.

Single character input function - putc()
This function operates on file that is **copied in writing mode**, which writes a single character to the file.

Syntax: putc(c,fptr);

Single character output function - getc()
This function operates on file that is **opened in reading mode.** Reads a single character from the file.

Syntax: getc(fptr);

Integer oriented file input and output functions for file

putw and getw are integer oriented functions. They are similar to the getc and putc functions and are used to read and write an integer.

Integer input function - putw()
This function is used to **write integer value on file** pointed by file pointer.

Syntax: putw(integer,*fptr);*

Integer output function - getw()
Reads an integer value from file pointed by file pointer.Returns next integer from input output stream.

Syntax: getw(*fptr);*

String input/output functions for file:

String input function – fgets()
The get(s) function is used to receive a string from the standard input device until a new line or EOF is encountered.

- Where the variable refers to the previously declared character array.
- The string received by the gets() function may include white space character.

- The gets() function will terminate only with a new line character.
- ***Note:***gets() function can receive data for only one variable.

  Syntax: fgets();

String output function – fputs()

The puts function is used to display a string at a time in the standard output device followed by a new character. Puts function accepts only a single argument.

Syntax: intfputs(char *s, fptr);

Mixed datai/o functions for file

The function **printf()** and **scanf()** are used for performing Input/Output operations in the file (Without involving file). Similar to scanf() and printf() functions.

(i) File input function -fscanf()

fscanf()is used to perform input operations in files.

fscanf()SYNTAX:	fscanf(fptr, ”format_string”, list_of_arguments);
Example	fscanf(fptr, ”%s %d %s”, &empname, &empno, &deptname);

(ii) File output function -fprintf()

fprintf()is used to perform Output operations in files.

fprintf()SYNTAX:	fprintf(fptr, ”format_string”, list_of_arguments);
Example	fprintf(fptr, ”%d %s”, empno, empname);

Unformatted read and write functions

The fread() and fwrite() functions often referred to as unformatted read and write functions. Similarly, files of this type are often referred to as unformatted files.

The fread() function is used for reading an entire block from a given file. The fwrite() function is used for writing an entire structure block to a given file.

fread()

This function **reads data from stream** of any data type.

Syntax :size_tfread(void *ptr, size_tsize,size_t n, FILE *fp);

size_t used for memory objects and repeats count.

fwrite()

This function writes to a stream or specified file. The syntax for fwrite() is

Syntax :size_tfwrite(const void *ptr,size_tsize,size_tn,FILE *fp);

where n is the number of elements written to that file.

Error handling while reading and writing files

The incorrect reading and writing situation are listed below:

1. Reading a file beyond the end-of-file (EOF).
2. Accessing a file that has not been opened.
3. Opening a file with the invalid filename.
4. Reading a file that is opened in write mode.
5. Writing on a file that is opened in read mode.
6. Tyring to write to a file which is write-protected one.
7. Closing a file that is not opened.

The feof() function:

The **feof()** function is used to check whether the file pointer is at the end-of-file or not. It takes the file pointer as the argument and returns non-zero if the file pointer is at the end-of-file. Otherwise it returns zero.

The ferror() function:

The **ferror()** function reports the status of the file indicated. It also takes a FILE pointer as its only argument and returns a non zero integer if an error has been detected up to that point, during processing. It returns zero otherwise.

10.Elucidate the following: (May 2018)

a) **fseek() function**
b) **rewind() function**

fseek() FUNCTION

The fseek() function allows us to read from or write to any point in the stream of bytes opened by using the fopen() function. It is used to move the position of the file pointer to the desired location in the file.

Syntax	fseek(fptr, offset, position)

Description	**fptr–**file pointer of the file to be accessed. **offset–**is a negative or positive number or a variable of long integer data type to reposition the file pointer towards the backward or forward direction from the location specified by position. **position–**Current position of the file pointer.
Example:	Beginning ↓ fseek(fptr,5L,0); ↑ 5 byte

rewind() FUNCTION

Function resets the file pointer **to the beginning** of the file.

***Syntax*:** rewind(fptr);

Example of rewind() function

```
#include<stdio.h>
void main()
{
 FILE *fp;
charch;
fp = fopen("file.txt","at+");           //Statement   1
if(fp == NULL)
            {
             printf("\nCan't open file or file doesn't exist.");
             exit(0);
            }
printf("\nWrite some data :\n");
while((ch=getchar())!=EOF)          //Statement   2
fputc(ch,fp);

rewind(fp);                             //Statement   3
printf("\nData in file :\n");
while((ch = fgetc(fp))!=EOF)          //Statement   4
printf("%c",ch);
fclose(fp);
      }
```

Output :

```
Write some data:
        Today is sunday.
        Data in file :
        Today is sunday.
```

11. Write a C program to generate a student mark sheet and store the information in a file. (Nov 2018)

```
#include <stdio.h>
int main()
 {
        char name[50];
        intmarks,i,n;
        printf("Enter number of students: ");
        scanf("%d",&n);
        FILE *fptr;
        fptr=(fopen("C:\\student.txt","w"));
        if(fptr==NULL)
{
                printf("Error!");
                exit(1);
        }
        for (i=0;i<n;++i)
{
                printf("For student%d\nEnter name: ",i+1);
                scanf("%s",name);
                printf("Enter marks: ");
                scanf("%d",&marks);
                fprintf(fptr,"\nName: %s \nMarks=%d \n",name,marks);
        }
        fclose(fptr);
        return 0;
    }
```

Output:

```
Enter number of students : 4
For student1
Enter name :codez
Enter marks : 88
For student2
Enter name : club
Enter marks : 75
For student3
Enter name :john
Enter marks : 66
```

```
For student4
Enter name : max
Enter marks :99
Process returned  0
```

13. Read a file and convert all small letters vowels into capital letters and store in another file. (Nov 2019)

```
#include
int main()
{
char string1[255];
int  i;
printf("Input a sentence: ");
gets(string1);
printf("The original string:\n");
puts(string1);
  i=0;
while(string1[i]!='\0')
   {
      if(string1[i]=='a' ||string1[i]=='e' ||string1[i]=='i' ||string1[i]=='o' ||string1[i]=='u')
     string1[i]=string1[i]-32;
      i++;
   }
printf("After converting vowels into upper case the sentence becomes:\n");
puts(string1);
}
```

OUTPUT :

```
Input a sentence : The original string :
international
After converting vowels into upper case the sentence becomes:
IntErnAtIOnAl
```